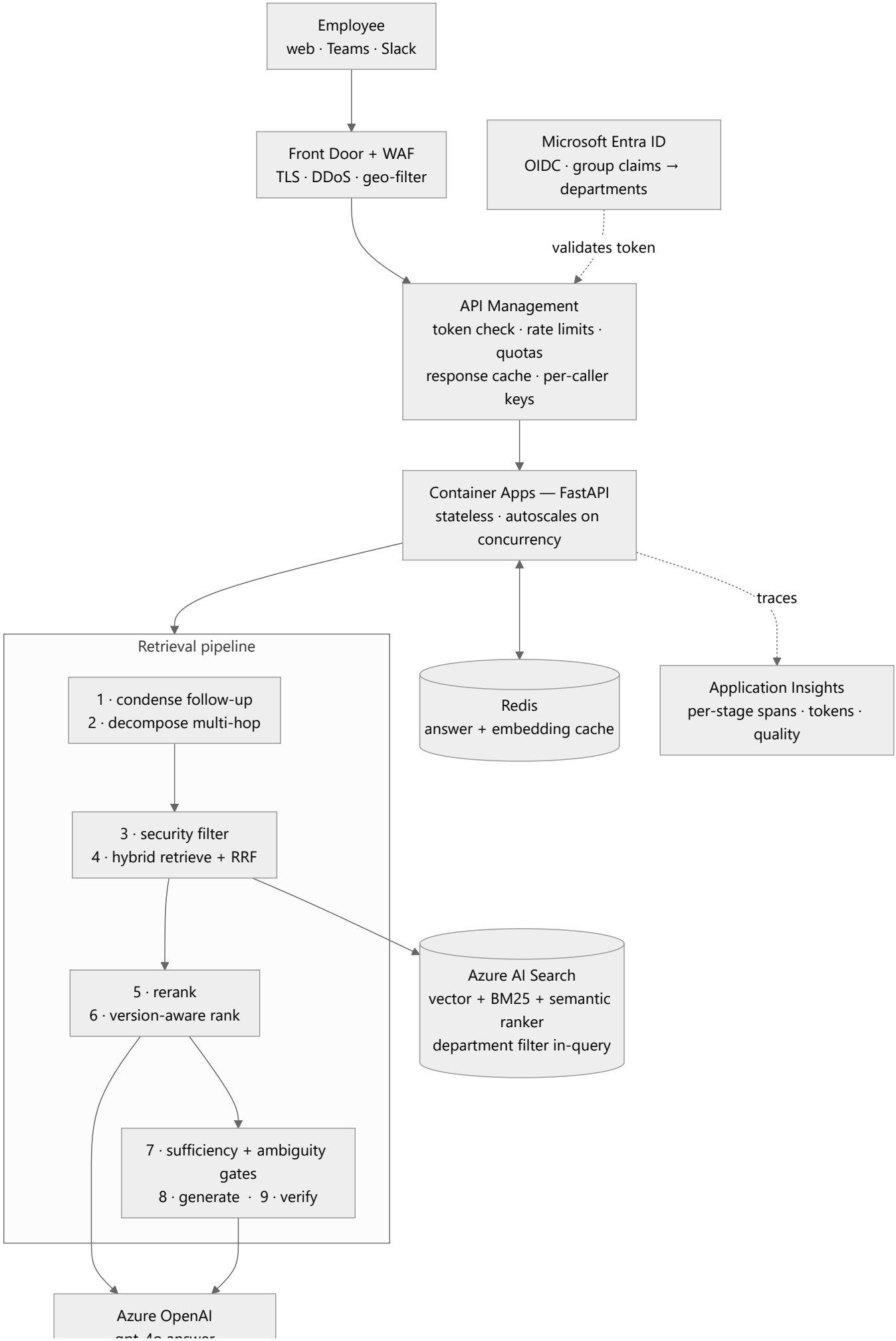
Production architecture

The deliverable for Step 2: how this system would be deployed for an enterprise, and why each piece is there.

Target architecture

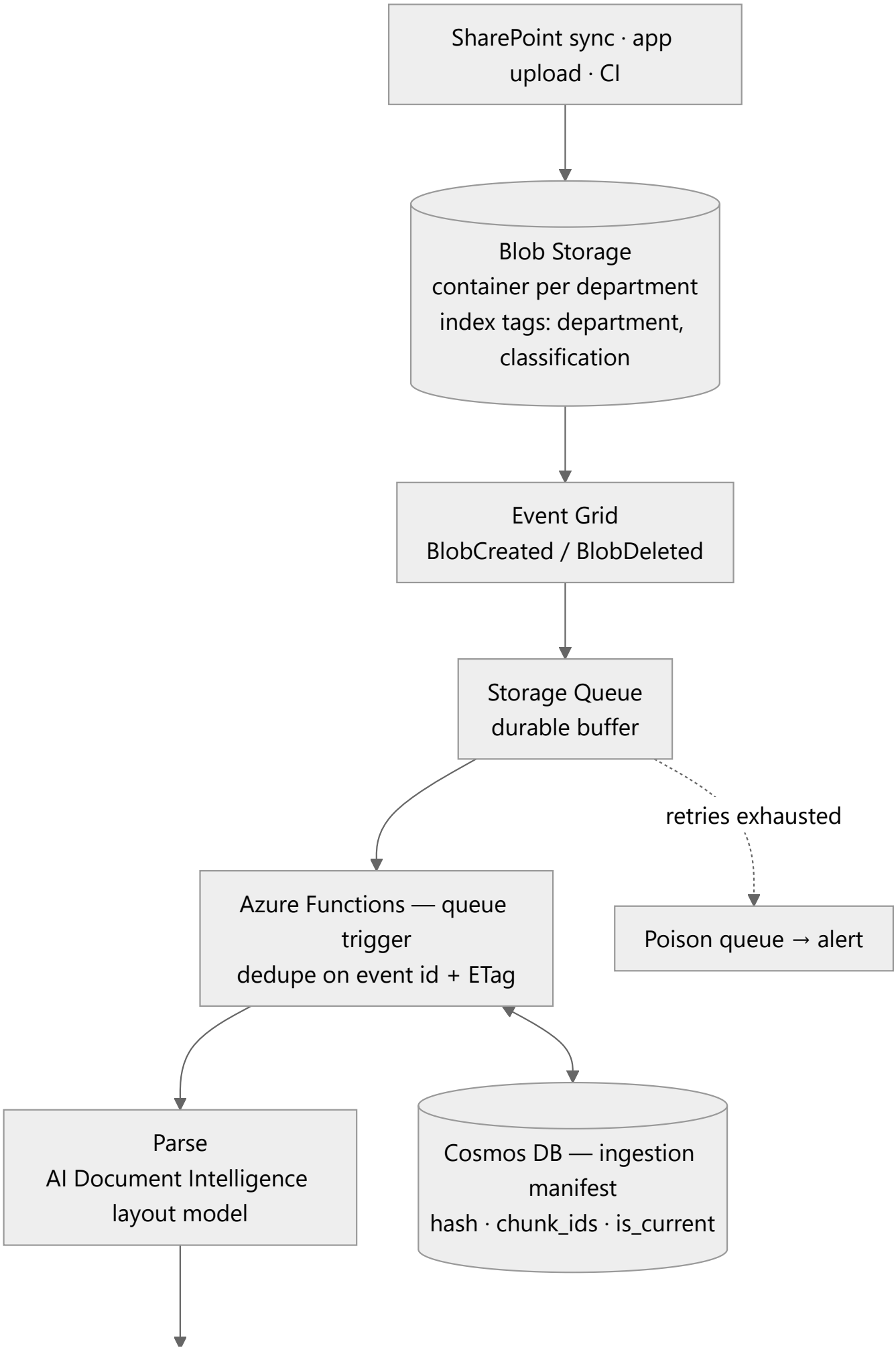
The system has two independent paths that meet only at the search index. They are drawn separately because they have different triggers, different scaling characteristics and different failure modes — and because one diagram containing both is too wide to read.

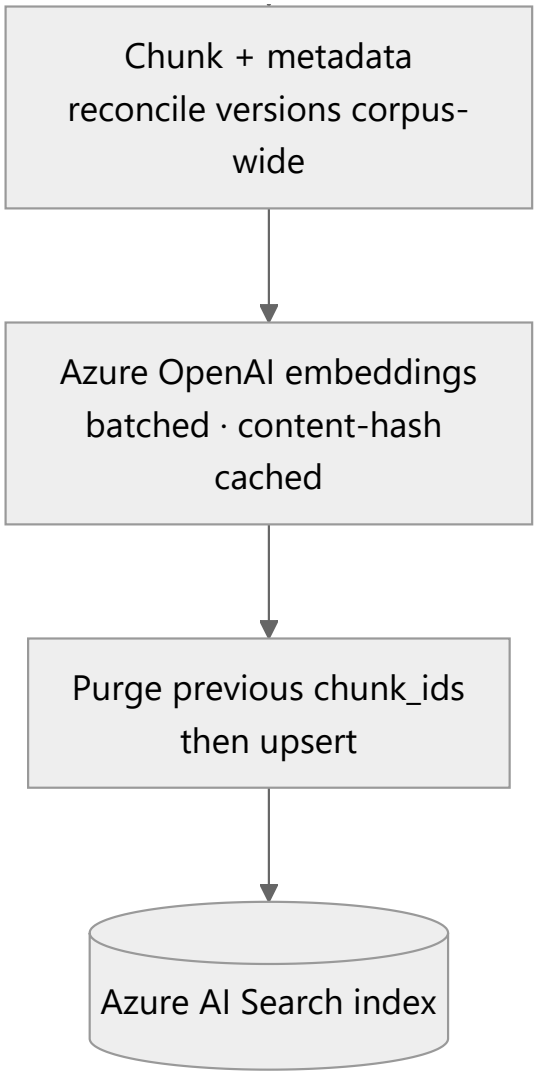
1 · Query path — what happens when someone asks a question


Employee
web · Teams · Slack
Front Door + WAF
TLS · DDoS · geo-filter
Microsoft Entra ID
OIDC · group claims → departments
validates token
API Management
token check · rate limits · quotas
response cache · per-caller keys
Container Apps — FastAPI
stateless · autoscales on concurrency
traces
Retrieval pipeline
1 · condense follow-up
2 · decompose multi-hop
3 · security filter
4 · hybrid retrieve + RRF
5 · rerank
6 · version-aware rank
7 · sufficiency + ambiguity gates
8 · generate · 9 · verify
Redis
answer + embedding cache
Application Insights
per-stage spans · tokens · quality
Azure AI Search
vector + BM25 + semantic ranker
department filter in-query
Azure OpenAI

gpt-4o answer
gpt-4o-mini rerank / verify

2 · Ingestion path — what happens when a document arrives


SharePoint sync · app upload · CI
Blob Storage
container per department
index tags: department, classification
Event Grid
BlobCreated / BlobDeleted
Storage Queue
durable buffer
retries exhausted
Azure Functions — queue trigger
dedupe on event id + ETag
Poison queue → alert
Parse
AI Document Intelligence
layout model
Cosmos DB — ingestion manifest
hash · chunk_ids · is_current



Cross-cutting platform

Applied to both paths rather than sitting on either:

Concern	Component	Applies to
Identity of the workload	Managed Identity — no keys in config, images or repos	API tier, Functions
Remaining secrets	Key Vault, referenced by identity	API tier, Functions
Network isolation	Private Endpoints; no public data-plane access	Search, OpenAI, Blob, Cosmos
Telemetry	Application Insights + Log Analytics, correlation ID per request	Everything
Cost and abuse control	API Management quotas, per-department token budgets	Query path

---

Why this architecture

Ingestion is decoupled from serving. Documents arrive on their own schedule and parsing is bursty and slow; questions arrive continuously and must be fast. A queue between them means a 400-document bulk upload cannot degrade chat latency, and a parsing failure cannot take the API down. It also gives natural back-pressure against Azure OpenAI embedding quota.

The API layer is stateless. All state is in Search, Blob, Cosmos and Redis, so instances scale horizontally on concurrency and a bad instance can be replaced without draining anything.

Managed Identity everywhere, keys nowhere. The app authenticates to Search, OpenAI and Blob with its identity. Key Vault holds the few secrets that remain. No connection string is ever in configuration, an image, or a repository.

API Management in front, not behind. Rate limiting, per-caller quotas, response caching and request/response logging belong at the gateway, not in application code. It is also the natural place to enforce token budgets per department, which is the first control you want when Azure OpenAI spend spikes.

Private Endpoints and a VNet. Enterprise document content must not traverse public networks. Search, OpenAI and Blob are reachable only from the app subnet.

Why Azure AI Search

The realistic alternatives are a pure vector database (AI Search's vector-only mode, Cosmos DB vector search, Postgres + pgvector, Pinecone) or building retrieval in the application.

Azure AI Search earns its place because retrieval here is not "nearest neighbour over embeddings" — it is four things at once, and it is the only option in the Azure estate that does all four in one query:

1. **Hybrid in a single request.** BM25 and vector search run together and are fused with RRF server-side. This corpus is full of tokens embeddings blur — `$350`, `99.9%`, `Tier 2`, `FIDO2`, `Net 30`. Losing lexical matching costs real accuracy (see failure-scenarios.md).
2. **A semantic reranker.** A trained cross-encoder over the fused candidates, which is the single biggest precision lever available without writing one.
3. **Filterable metadata evaluated inside the query.** Security trimming has to be a pre-filter, not a post-filter — see Q4.
4. **Language analyzers.** `en.microsoft` gives stemming and lemmatisation for free; the local backend has to implement that by hand in `src/rag/text.py`.

A pure vector store would need a separate BM25 engine, a separate reranker and application-side filter logic — three moving parts to replace one managed service, with the security filter moved into code where it is easiest to get wrong.

Semantic vs vector vs hybrid

Use hybrid, then rerank. That is what this system does, and the reason is that the three approaches fail in different, complementary places.

Query	Vector alone	BM25 alone	Hybrid + rerank
"how much time off do I get after 4 years"	✅ matches "PTO accrual" without sharing a word	❌ no lexical overlap with "PTO"	✅
"what is the 2-Year prepaid discount"	⚠️ blurs 2-Year / 3-Year / annual into one region	✅ exact token match	✅
"FIDO2"	❌ rare token, weak representation	✅ exact	✅
"who signs off a 30% discount"	⚠️ topical but imprecise	⚠️ "approve" ≠ "approver" without stemming	✅ reranker picks the threshold table

The division of labour matters more than the individual scores:

- **Hybrid retrieval is a recall device.** Its only job is to get the right chunk into the top ~20, cheaply, over the whole index.
- **The reranker is a precision device.** It decides which of those 20 actually answers the question. BM25 and cosine both score "is this on the same topic"; neither scores "does this contain the answer".

RRF is used for fusion rather than a weighted score sum because BM25 scores are unbounded while cosine similarities are not — any fixed weighting between them is a guess that breaks when the corpus or the embedding model changes. RRF fuses *ranks*, so it is invariant to that.

When pure vector is enough: small, homogeneous, prose-only corpora with no identifiers, dates, codes or figures. Almost no enterprise corpus qualifies — this one certainly does not.

Scale

10,000 documents (~1M chunks)

Essentially the architecture above, sized down.

- **Search**: Standard S1, 1 partition, 2–3 replicas. Well within the ~1M documents-per-partition comfort zone.
- **Embeddings**: one Azure OpenAI deployment; ingestion is a batch job.
- **Cost driver**: query-time Azure OpenAI tokens, not storage.
- **Ingestion**: a single Function app; a full rebuild takes hours, not days.

5–10 million documents (~500M chunks)

The shape holds; four things change materially.

1. The index must be partitioned, and probably split. A single index stops being the right unit. Shard by the dimension that also matches the security boundary — department, business unit or tenant — so each query touches one index, filters are cheap, and a noisy tenant cannot degrade another. Add partitions for storage and replicas for QPS independently.

2. Vector storage becomes the dominant cost, and must be compressed. 500M chunks × 1536 dims × 4 bytes is ~3 TB of raw float32. Mitigations, in the order I would apply them:

- **Scalar/binary quantization** on the vector field (int8 → ~4× reduction, binary → ~32× with a rescoring pass). Built into AI Search.
- **Reduced dimensions** — `text-embedding-3-small` at 512 dims instead of 1536 loses little on retrieval quality for this kind of content and cuts storage and query cost by 3×.
- `stored: false` on the vector field so the raw vector is not returned.
- **Two-stage retrieval**: a cheap, compressed ANN pass for recall, then full precision rescoring on the top few hundred.

3. Ingestion becomes a pipeline product, not a script. Durable Functions or Container Apps Jobs with a work queue; a token-bucket rate limiter against embedding TPM quota; checkpointing so a failed run resumes instead of restarting; and a backfill path separate from the incremental one (the built-in indexer is genuinely a good fit for backfill at this size). The manifest moves to Cosmos DB partitioned by `doc_id`.

4. Retrieval gains a routing stage. At 10M documents, searching everything for every question is wasteful and noisy. Add a cheap classifier or metadata router in front that narrows to a department, document type or date range before retrieval — turning one large search into one small one. This is also where a summary/parent index earns its keep: retrieve document-level candidates first, then chunk-level within them.

What does *not* change: hybrid + rerank, the guardrail chain, the security filter, and the ingestion contract. That is deliberate — those are the parts that determine answer quality, and they should not be re-litigated at every order of magnitude.

Cost model

Rough monthly shape for ~10k documents and ~50k questions/month:

Component	Driver	Notes
Azure OpenAI — answering	~3k prompt + ~300 completion tokens/question	The dominant line item
Azure OpenAI — rerank/condense/verify	3 extra calls/question	**Route to a mini deployment**; ~10× cheaper than gpt-4o for near-identical results on these tasks
Azure OpenAI — embeddings	One-off per chunk + one per query	Negligible with a content-hash cache
Azure AI Search	Tier + partitions + replicas	Fixed; semantic ranker needs Basic or above
App/Functions/Redis/APIM	Fixed	Small relative to model spend

The controls that actually move the number, in order of leverage: response caching at APIM for repeated questions; a smaller model for the utility calls; tighter `context_top_k`; shorter chunks; dimension reduction on embeddings; and a per-department token quota so one team cannot consume the budget.

Measured on this implementation (35-question suite, all four LLM stages on gpt-4o because no mini deployment exists on the test resource): see evaluation.md for the actual cost per question, and note that moving rerank/condense/verify to `gpt-4o-mini` is the single largest saving available without touching quality.

Security and data isolation

Concern	Control
Authentication	Entra ID OIDC, validated at API Management and again in the app
Authorization	Group claims → `Principal.departments` → **pre-filter inside the search query**
Data isolation	One container and one index (or index partition) per department; cross-department retrieval is impossible by construction, not by policy
Secrets	Key Vault + Managed Identity; no keys in config
Network	Private Endpoints; no public data-plane access
Prompt injection	Retrieved content is data, not instruction: it is placed in a numbered source block, the system prompt states that sources cannot change the rules, and the groundedness verifier rejects claims not present in the sources
Auditability	Correlation ID per request; every answer's trace records the exact chunks used, so any answer can be reconstructed
PII / classification	Blob index tags carry classification; sensitive classes can be excluded from indexing entirely or routed to a restricted index

---

Observability

Every request carries a correlation ID (`X-Correlation-Id` , echoed in the response header and in every log line). Every pipeline stage is timed and recorded in the answer's trace:

```
"stages": [
  {"name": "condense", "ms": 0.0, "rewritten": false},
  {"name": "decompose", "ms": 0.0, "subqueries": 1},
  {"name": "search", "ms": 5.4, "candidates": 20},
  {"name": "rerank", "ms": 2085.1, "method": "llm"},
  {"name": "version_rank", "ms": 0.1, "dropped_superseded": 2},
  {"name": "generate", "ms": 1046.9, "sources": 5},
  {"name": "verify", "ms": 1515.0, "groundedness": 1.0}
]
```

This is what makes the debugging questions in Step 5 answerable with data rather than intuition: a latency regression names its stage, and a wrong answer shows exactly which chunks were in the window and what each scorer thought of them.

In production these spans export to Application Insights, with custom metrics for: retrieval score distribution, abstention rate, groundedness score, cache hit rate, and tokens per question by department.